1 Design

1.1 Structure

The shared support runtime provides mutual TLS, identity-token binding, ConsumerAuthorization checks, registration, telemetry and the JSON state. `testtranslation.go` adds the interface translator, the data model translator and the bridge data plane (route `/testtranslation/bridge/{bridgeId}/{path...}`, access `bridge`).

1.2 Bridge lifecycle

1. `check-targets`: returns the targets having a `generic_https` interface (policy `CERT_AUTH` or `NONE`) with an access address, port and the requested operation.
2. `initialize-bridge`: stores the bridge (operation method and path template, target interface properties, input and result data model translator descriptors) and answers the bridge interface: `generic_https`, policy `TRANSLATION_BRIDGE_TOKEN_AUTH`, base path `/testtranslation/bridge/<bridgeId>`, the operation, and `dataModels` of the consumer.
3. A bridge call: token and operation check; the body is translated by the input data model translator (`init-translation`, then `get-translation-result`); the request (query string included) is forwarded to the target with TestTranslation's identity; a successful result is translated by the result data model translator; `USED` is reported. An unreachable target reports `EXTERNAL_ERROR` and ends the bridge; a failed translation reports `INTERNAL_ERROR`.
4. `abort-bridge/<bridgeId>`: only the system that initialized the bridge may abort it; the bridge stops forwarding (404).

1.3 Persistence

Bridges: JSON state (`bridges`). Translation tasks: memory. No secrets are stored.

1.4 Important delimitations

TestTranslation is a test application system for the TranslationManager, not a production translator.

- One bridge kind: `generic_https` to `generic_https`, one negotiated operation per bridge; targets with `CERT_AUTH` or `NONE` policies (authorization tokens for token-protected targets are not presented).
- Data models `simpleJson` and `senml` only; translations are synchronous (a task is `DONE` or `ERROR` when `init-translation` returns).
- Bridges are kept in the system's JSON state; translation tasks only in memory (latest 1000).